

Evaluating the Security of AI Generated Authentication Code:

A Comparison of LLMs Under Naive and Security Aware Prompts

CS 544 Final Report — Spring 2026

Jordan Martinez

Department of Computer Science, University of New Mexico

Instructor: Afsah Anwar

Abstract

AI code generation tools are being increasingly used in software development, but the security of the code they generate is not quite understood across different tools or prompting styles. This project investigates whether AI generated authentication code introduces security vulnerabilities and whether how specific the prompt is affects the security of the generated code. I compared three large language models (ChatGPT, Gemini, and Claude Code) on five Flask authentication tasks (user registration, password storage, login endpoint, session management, and password reset), using two prompt styles per task: a naive prompt with no security guidance mentioned, and a security aware prompt that explicitly requested security. Each of the generated 30 code samples were analyzed using Bandit, an automated static analysis tool, and then reviewed manually against an OWASP-based checklist. Across all samples, Bandit identified 21 findings while manual review identified 114, a $5.4\times$ gap that highlights the limits and/or concern of just using static automated analysis for evaluating AI generated code. Security aware prompting reduced the total number of findings for every LLM, but the size of the reduction varied: ChatGPT improved most (50% fewer findings), followed by Claude Code (29%) and Gemini (24%). Across both prompt styles combined, ChatGPT produced the fewest findings overall (36), followed by Claude Code (41) and Gemini (58), though Claude Code had zero Bandit findings which would have suggested a stronger result if static analysis had been the only evaluation method. Cross file inconsistencies within a single LLM's output, where one file used secure patterns and another did not, appeared across all three tools and were invisible to static analysis. The results show that prompt specificity improves AI generated code security, but that manual review is still essential for catching the design level security issues that static analysis cannot detect.

1 Introduction

AI powered code generation tools like ChatGPT, Google Gemini, and Claude Code have become a regular part of how developers create software, but the security of the code they produce is not understood across different tools or prompting styles. Prior work has shown that AI generated code often contains vulnerabilities: Pearce et al. (2022) found that roughly 40% of GitHub Copilot's suggestions contained security weaknesses across 89 scenarios, Khoury et al. (2023) found that only 5 of 21 ChatGPT generated programs met basic security standards, and Siddiq et al. (2024)

used **Bandit** and other static analyzers to identify quality and security issues in ChatGPT generated Python code from real developer conversations with the model. What is less clear is whether the way a developer phrases their prompt changes the security of the result, and whether different LLMs produce code of comparable security on the same task. This project asks: *Does prompt specificity meaningfully affect the security of AI generated authentication code, and how do different LLMs compare when given identical prompts?* I generated 30 Flask authentication code samples across three LLMs (ChatGPT, Gemini, Claude Code) and two **prompt styles** (a naive prompt with no security guidance and a security aware prompt that explicitly requested secure coding practices), then evaluated each sample using both Bandit static analysis and a manual code review drawn from the **OWASP Top 10**. The five tasks (user registration, password storage, login endpoint, session management, password reset) cover the highest risk parts of an authentication system. By holding tasks, prompts, and analysis method constant across all three LLMs, the comparison highlights the effect of model choice and prompt style on the security of the generated code.

1.1 Contributions

My project makes three main contributions. First, I show that prompt specificity meaningfully improves the security of AI generated authentication code, but the effect varies by LLM: ChatGPT showed a 50% reduction in findings under security aware prompts, Claude Code 29%, and Gemini 24%. Second, I show that Bandit static analysis alone is insufficient for evaluating the security of AI generated code. Bandit identified 21 findings across the 30 sample dataset while manual review identified 114, a $5.4\times$ gap. Third, I document a pattern of **cross file inconsistency** in AI generated code, where the same LLM produces secure patterns in one file and insecure patterns in another file of the same project. This inconsistency was invisible to static analysis and appeared across all three LLMs, representing a real risk for developers using these tools prompt by prompt to build larger systems.

2 Methodology

2.1 Task Selection

Five authentication tasks were selected to cover the highest risk surfaces of a typical login system: user registration, password storage, login endpoint, session management, and password reset. These tasks were chosen because they map to multiple categories of the OWASP Top 10, including A02 (Cryptographic Failures), A03 (Injection), A05 (Security Misconfiguration), and A07 (Identification and Authentication Failures) [4]. Each task represents a distinct piece of an authentication system that a developer might reasonably ask an AI tool to generate in isolation. The project was originally going to include rate limiting as a sixth task, but this was moved to future work as I began to run out of time with the tedious manual reviews.

2.2 Prompt Design

For each task, two prompts were written: a **naive prompt** that described the desired functionality without mentioning security, and a **security aware prompt** that added explicit requirements for secure coding practices appropriate to the task. The naive prompts were written to sound like something someone in a hurry might type, with no security keywords. The security aware prompts named specific security concerns (for example, SQL injection protection, secure password hashing, anti-enumeration, or single-use tokens) without giving implementation details, so that the LLMs had to generate the requirements into code themselves. Both prompts for each task were held constant across all three LLMs to enable fair comparison. The full text of all ten prompts is included in the project repository under `prompts/`.

2.3 LLMs and Generation Procedure

Three large language models were compared: **ChatGPT** (OpenAI) [9], **Gemini** (Google) [10], and **Claude Code** (Anthropic) [11]. ChatGPT and Gemini were accessed through the web using default model settings, which is in line with the typical experience rather than a maxed out configuration. Claude Code was used with 'default effort level', again to reflect typical usage rather than a maxed config.

Each of the ten prompts ($5 \text{ tasks} \times 2 \text{ prompt styles}$) was submitted to each of the three LLMs in a fresh session with no prior conversation in each, producing a total of 30 code samples. Each generated file was saved to the project repository with a docstring header recording the LLM used, the prompt style, the date generated, the full prompt text, and any observations about the generation process (such as clarifying questions or unusual model behavior). Generated code was preserved exactly as produced by each LLM with no modifications, since runnability was treated as a data point rather than properties to be corrected.

2.4 Automated Analysis: Bandit

Each of the 30 code samples were analyzed using **Bandit** version 1.9.4 [8], a static analysis tool that detects common security issues in Python code. Bandit was run via a shell script that produced a JSON report for each file, organized by LLM and prompt style. Findings were taken from these JSON reports and recorded in a results matrix, with each finding tagged by LLM, prompt style, task, severity (Bandit uses HIGH/MEDIUM/LOW), CWE ID, and Bandit test ID.

2.5 Manual Code Review

A manual code review was performed on each of the 30 samples using a structured checklist taken from the OWASP Top 10 [4], the OWASP Authentication Cheat Sheet [6], and the OWASP Password Storage Cheat Sheet [7]. The checklist covered seven categories of issues: SQL injection patterns, password hashing practices, input validation, session and token handling, error handling and information disclosure, rate limiting and brute force protection, and a final OWASP Top 10 summary. For each file, the review documented findings that Bandit had flagged (and whether they were real issues or false positives), additional findings that Bandit had missed, the OWASP

category each finding mapped to, and the corresponding CWE ID from the MITRE CWE catalog [5]. The full set of 30 manual review documents is included in the project repository under `analysis/manual_review/`.

2.6 Results Matrix

All of the findings from both Bandit and manual review were put into a CSV file in my repository at: `analysis/results_matrix.csv` with the columns: LLM, Prompt_Style, Task, Vulnerability_Category, Severity, Tool_Detected, CWE_ID, Description, and Notes. The *Tool_Detected* column distinguishes whether a finding came from Bandit or from manual review, which is the basis for the comparison in the Results section. A Python script (`analysis/generate_figures.py`) reads this CSV and produces the four figures presented in the Results section, making the analysis fully reproducible.

2.7 Use of AI in the Project Workflow

This project studied AI generated code, and AI assistance was also used in producing the project itself. Claude (Anthropic, via `claude.ai`) was used during the planning of the manual review structure, and creating the figures based on my results matrix [12]. I made all final judgments about the findings, interpretations, and final results. One methodology note specific to Claude Code is because Claude Code operates as an agentic model that writes files directly rather than returning code that you copy and paste, it sometimes populated the docstring `Notes` field of its own generated files with its own description of what the code did, rather than leaving the field for researcher observations like the ChatGPT and Gemini samples. This inconsistency does not affect the code being analyzed, but is documented here for transparency. The use of AI is important for me to mention here in keeping with the project’s theme of transparency around AI generated work.

3 Results

The analysis produced 135 total findings across the 30 code samples: 21 detected by Bandit static analysis and 114 detected through manual code review. This section presents the findings through four figures, each examining the data from a different POV, followed by a summary of the patterns that emerged across all 30 samples.

3.1 Bandit vs. Manual Review by LLM and Task

Figures 1 and 2 present heatmaps of findings by LLM, prompt style, and task. The two heatmaps share a common color scale to allow direct visual comparison between the automated and manual analysis results.

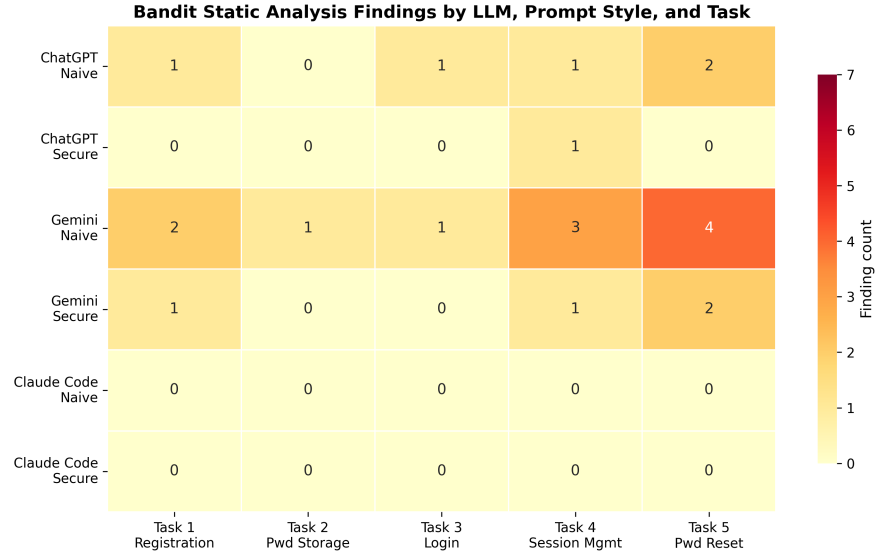


Figure 1: Bandit static analysis findings by LLM, prompt style, and task. Cells show the number of findings Bandit identified for each combination.

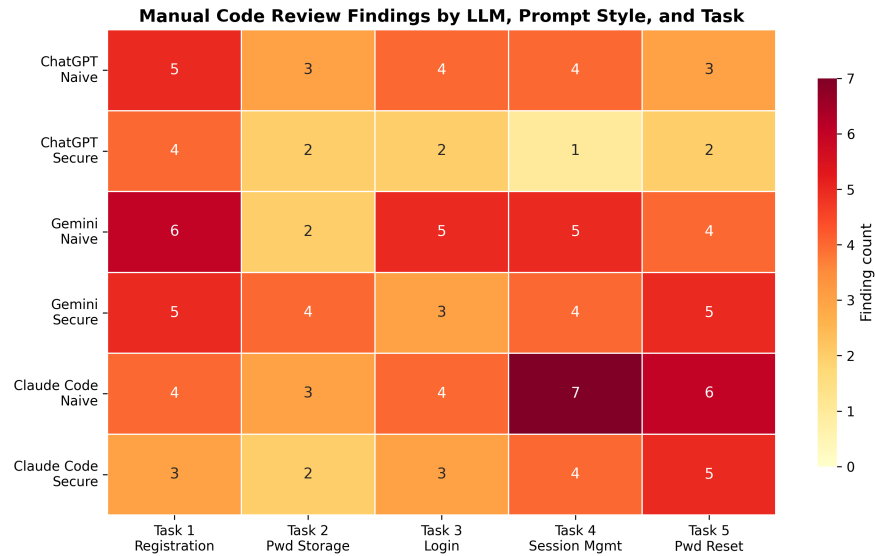


Figure 2: Manual code review findings by LLM, prompt style, and task, using the same color scale as Figure 1. Manual review identified findings in every cell, including the cells where Bandit reported zero issues.

The contrast between the two heatmaps is very apparent. The Bandit heatmap is mostly light colored, with most cells showing zero or one finding, and the highest single cell at 4 findings (Gemini naive, password reset). Claude Code’s two rows are entirely empty in the Bandit heatmap. The manual review heatmap, by contrast, has findings in every cell, with several cells in the 4–7 range. The cell with the highest count in the entire dataset is Claude Code naive Task 4 (session management), with seven manual findings, the same cell that Bandit reported as having zero findings.

This direct comparison is the central argument of the project. Bandit’s detection level does not overlap with the kinds of vulnerabilities present in this dataset. The tool catches very little when compared to the manual look at things. It catches things such as Flask’s `debug=True` flag and hardcoded credential patterns, but is blind to the design level flaws that were present via manual review findings: missing rate limiting, missing session fixation protection, plaintext password comparisons, weak token storage, and information leakage through logging.

3.2 Findings by Detection Method

Figure 3 shows the heatmap data put into a single chart comparing Bandit findings against manual review findings for each of the six LLM prompt style groups.

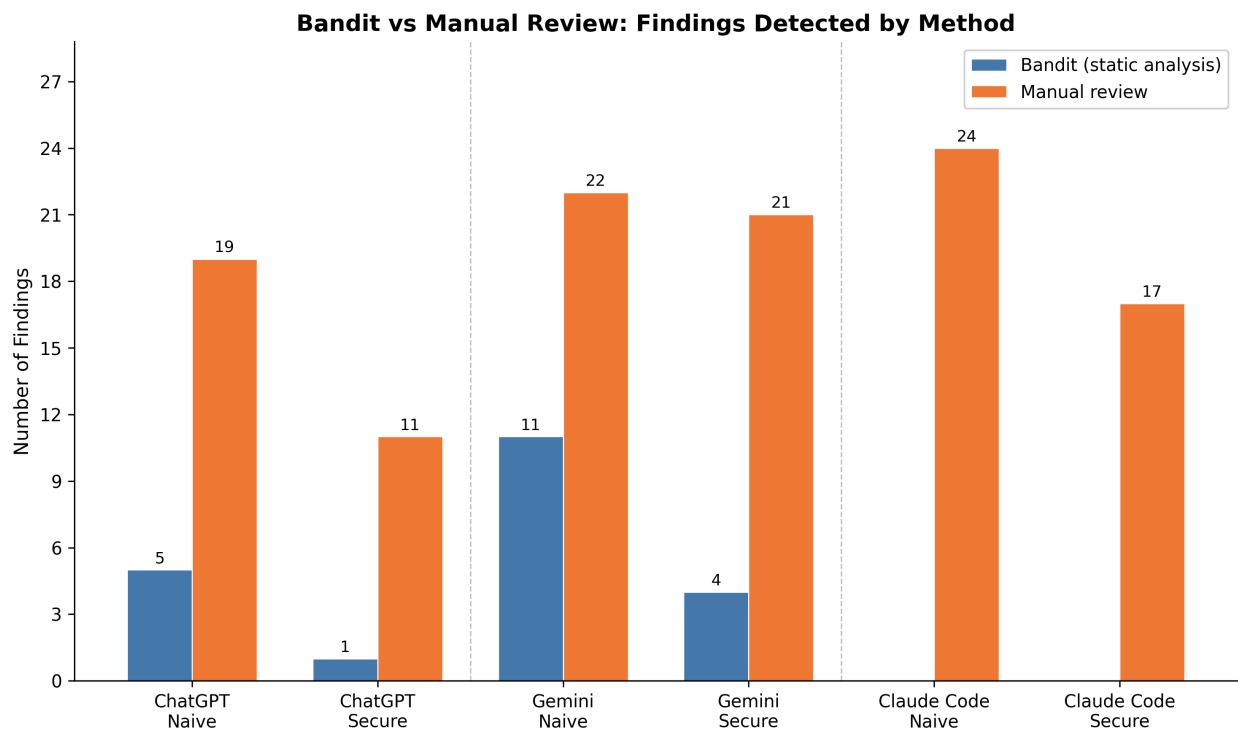


Figure 3: Comparison of findings detected by Bandit static analysis (blue) and manual code review (orange) across each LLM and prompt style.

Bandit identified 21 findings across the full dataset, while manual review identified 114, a $5.4\times$ gap. The gap exists in every LLM prompt group, but its amount changes. For Gemini naive, Bandit caught 11 of 33 total findings (33% detection rate), the highest rate of any group. For ChatGPT naive, Bandit caught 5 of 24 findings (21%). For Claude Code naive, Bandit caught 0 of 24 findings (0%). Manual review found 24 issues in the Claude Code naive samples, comparable to the 19 found in ChatGPT naive and the 22 found in Gemini naive. The zero rate is because Claude Code did not produce the specific patterns Bandit looks for, such as Flask debug mode or hardcoded placeholder credentials. The underlying vulnerabilities were present, just not in the form Bandit can recognize.

3.3 Effect of Prompt Specificity on Findings

Figure 4 shows the total number of findings per LLM prompt style group, broken down by severity level (Critical, High, Medium, Low).

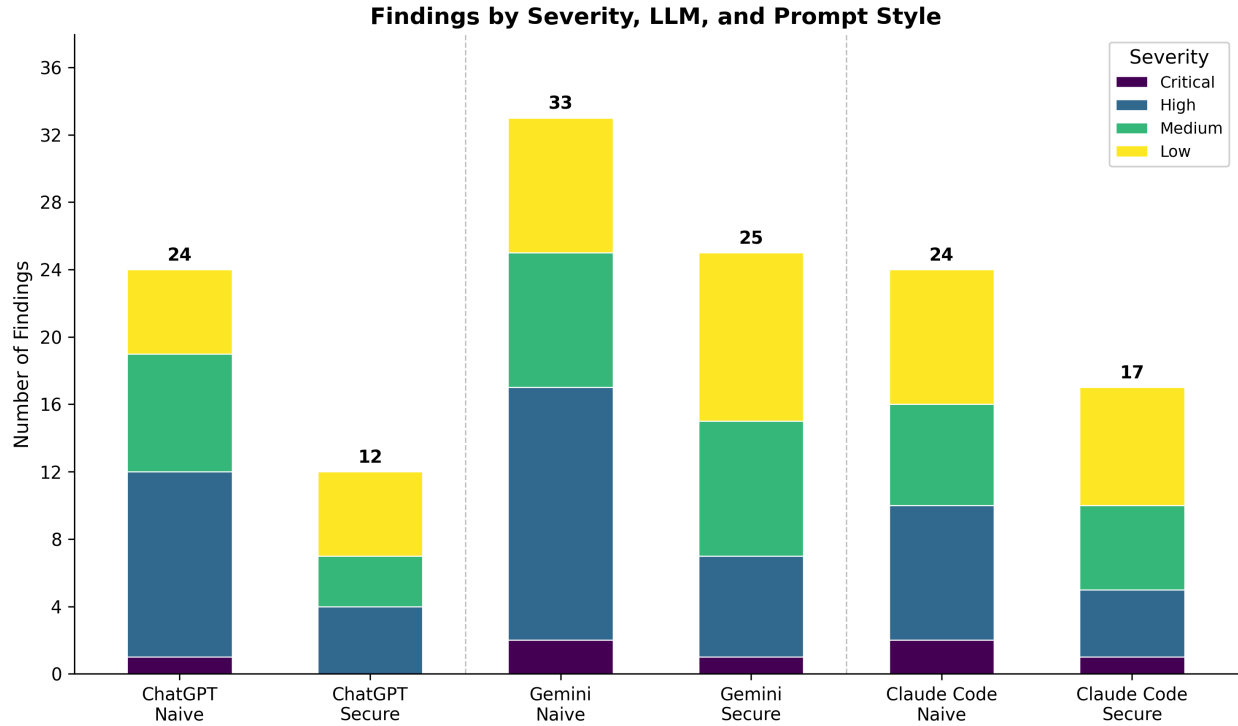


Figure 4: Total findings per LLM and prompt style, broken down by severity. Every LLM produced fewer total findings with security aware prompts than with naive prompts.

Security aware prompting reduced the total number of findings for every LLM, though by different amounts. ChatGPT showed the largest improvement: from 24 findings under naive prompts to 12 under security aware prompts (50% less). Claude Code showed the second largest improvement: from 24 findings to 17 (29% less). Gemini showed the smallest: from 33 findings to 25 (24% less). Across all three LLMs combined, security aware prompting reduced total findings from 81 to 54, a 33% reduction.

The severity composition is also informative. Critical severity findings (the lowest segments of the bars) appeared in every group except ChatGPT secure, which is the only group with no 'Critical' findings. The critical findings in the dataset correspond to design level failures: plaintext password comparison in SQL queries (ChatGPT naive Task 3, Gemini naive Task 3), plaintext password storage in an in-memory dictionary (Claude Code naive Task 4), passwords hashed with SHA-256 instead of an adaptive algorithm (Claude Code secure Task 4), hardcoded credentials in a login route (Gemini naive Task 4, Gemini secure Task 4), and a reset link returned directly in a JSON response (Claude Code naive Task 5). Bandit detected only one of these seven 'Critical' findings (the hardcoded credentials in Gemini's Task 4, where the literal password string 'secret' appeared in the source code). The other six were found only through manual review.

3.4 Bare Minimum Output and Cross File Inconsistency in LLMs

Rate limiting was absent in 28 of 30 samples. Only Claude Code’s secure Task 3 implemented any form of rate limiting (an in memory sliding window rate limiter), and only ChatGPT’s secure Task 4 made any attempt at runtime defensive measures through session hardening. No prompt explicitly asked for rate limiting, and consequently no other sample included it. This pattern suggests that LLMs implement what the prompt literally requests and do not go above and beyond unless explicitly asked to enforce defensive practices. This is a key finding because it suggests that AI generated authentication code still depends on human security expertise, especially when prompts don’t include important defensive requirements.

Inconsistency within a single LLM’s output was common. ChatGPT’s naive Task 1 hashed passwords with `werkzeug`, but its naive Task 3 compared passwords in plain text in the SQL query. Claude Code used `bcrypt` correctly in naive Tasks 2 and 3, but reverted to a plaintext user store in naive Task 4 and SHA-256 hashing in secure Task 4. Gemini used `bcrypt` in naive Task 2 but plaintext password comparison in naive Task 3. In every case, the inconsistency would produce a non-functional or insecure authentication system if the files were combined into a single application without manual review.

4 Discussion and Future Work

The results suggest that LLMs implement what the prompt asks for and not much beyond that. Every security aware prompt produced code that addressed the security concerns named in the prompt, but rarely went further. Rate limiting illustrates this: it was missing from 28 of 30 samples, appearing only when the prompt nudged toward broader runtime defenses. For developers, this means prompts cannot be partial. A prompt that names SQL injection but not rate limiting will produce code that defends against the first and ignores the second. The cross-file inconsistency finding is the most concerning result for real world AI assisted development. Each generated file is internally coherent, but inconsistencies only become visible when files are read together. A developer building an authentication system prompt by prompt would have no signal that the files are incompatible until login broke at runtime. This is a category of bug that static automated analysis cannot catch, because it requires reasoning about the system as a whole rather than each file individually.

Based on the findings, I would rank ChatGPT as the most reliable of the three LLMs for AI assisted authentication code, with Claude Code second and Gemini third. ChatGPT consistently applied defensive practices like password hashing and parameterized queries even under naive prompts, and showed the largest improvement under security aware prompting. Claude Code produced strong individual files but suffered the most dramatic cross-file inconsistencies, while Gemini produced the highest total finding count and the most placeholder credential patterns that a careless developer might ship or not even notice.

This study has limitations. The sample size is small (30 code samples), the tasks are limited to Python Flask authentication, the prompts were authored by just myself, and manual review intro-

duces subjectivity. The LLMs were accessed in March-May 2026 using default model settings, and future versions may behave differently.

4.1 Future Work

Several directions would extend this work. The most direct is adding the auth surfaces cut from the original scope: rate limiting, logout, multi-factor authentication, and account lockout. A second direction is including additional LLMs like GitHub Copilot and Codex to test whether these patterns hold across more LLMs than the 3 I investigated. A third is deploying the generated code and testing exploitability live with tools like Wireshark, since manual analysis identifies potential vulnerabilities but live testing would see which are actually exploitable. A fourth is testing prompts authored by multiple developers to separate the effect of LLM choice from individual prompting style.

5 Conclusion

This project compared three AI code generation tools (ChatGPT, Gemini, and Claude Code) on five Flask authentication tasks under two prompt styles, evaluating the 30 code samples through both Bandit static analysis and manual code review. Prompt specificity meaningfully improved the security of the generated code for every LLM tested, but the size of the improvement varied substantially. Static analysis alone proved insufficient: Bandit caught only 21 of the 135 total findings, missing six of the seven critical severity issues in the dataset. The most concerning pattern was cross-file inconsistency, where the same LLM produced secure code in one file and insecure code in another file of the same project. For developers using AI tools to assist with authentication code, the takeaway is that prompt design matters, automated analysis is not enough on its own, and the system as a whole still needs human review even when each individual file looks correct.

References

- [1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in *Proc. 43rd IEEE Symposium on Security and Privacy (SP)*, San Francisco, CA, USA, 2022, pp. 754–768.
- [2] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, “How Secure is Code Generated by ChatGPT?” in *Proc. IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, Honolulu, HI, USA, 2023.
- [3] M. L. Siddiq, L. Roney, J. Zhang, and J. C. S. Santos, “Quality Assessment of ChatGPT Generated Code and their Use by Developers,” in *Proc. 21st International Conference on Mining Software Repositories (MSR ’24)*, Lisbon, Portugal, 2024.
- [4] OWASP Foundation, “OWASP Top 10:2021,” 2021. [Online]. Available: <https://owasp.org/Top10/>.

- [5] The MITRE Corporation, “Common Weakness Enumeration (CWE),” 2026. [Online]. Available: <https://cwe.mitre.org/>.
- [6] OWASP Foundation, “Authentication Cheat Sheet,” OWASP Cheat Sheet Series, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.
- [7] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.
- [8] PyCQA, “Bandit: A security linter for Python,” version 1.9.4, 2026. [Online]. Available: <https://bandit.readthedocs.io/>. [Accessed: April, 2026].
- [9] OpenAI, “ChatGPT,” 2026. [Online]. Available: <https://chat.openai.com/>. [Accessed: March, 2026].
- [10] Google, “Gemini,” 2026. [Online]. Available: <https://gemini.google.com/>. [Accessed: April, 2026].
- [11] Anthropic, “Claude Code,” 2026. [Online]. Available: <https://docs.claude.com/en/docs/claude-code/>. [Accessed: March, 2026].
- [12] Anthropic, “Claude (claude.ai),” 2026. [Online]. Available: <https://claude.ai/>. [Accessed: May, 2026]. Used for manual review structure, and figure design.